

Notices

- Assignment 1 is due on 17 April.
- Final Exam is “open book with restrictions:
 - Electronic notes are allowed (pdf, word).
 - GenAI is not allowed
 - Blank paper is allowed for scratches
- Any questions?

3008ICT Deep Learning

Week 6. Recurrent Neural Networks

- Binary Addition as RNN
- RNN Architecture
- Fixed-window RNN
- LSTM
- Bidirectional RNN (BRNN)

Adapted from slides at d2l.ai and Stanford CS22n

Binary Addition as RNN: Shortcoming of MLPs



Binary Addition

$$\begin{array}{r} 00110100 \\ 01001101 \\ \hline 10000001 \end{array}$$

← time

$$\begin{array}{r} 001101000 \\ 010011011 \\ \hline 100000001 \end{array}$$

← time

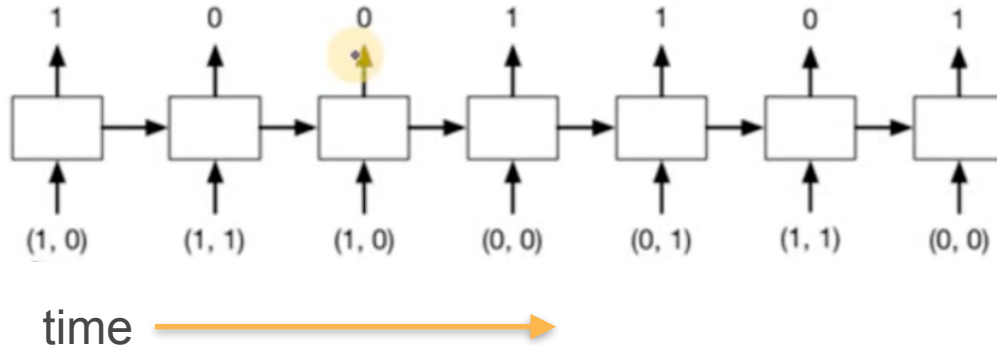
Shortcoming of MLP

- Recall: a MLP is a special feed-forward neural network.
- A feedforward network could be trained to do binary addition, but there are some obvious barriers:
 - We must decide in advance the maximum number of digits in each number.
 - The processing applied to the beginning of a long number does not generalise to the end of the long number because it uses different weights.
- As a result, feedforward nets do not generalise well on the binary addition task.

Binary Addition as RNN

Binary addition can be easily represented as a recurrent neural network (RNN)

- The network has two input units and one output unit.
 - It is given two input digits at each time step.
 - The desired output at each time step is the output for the column that was provided as input two time steps ago.
- It takes one time step to update the hidden units based on the two input digits.
–It takes another time step for the hidden units to cause the output.



$$\begin{array}{r} 0100111 + \\ 0110010 = \\ \hline 1011001 \end{array}$$

← time

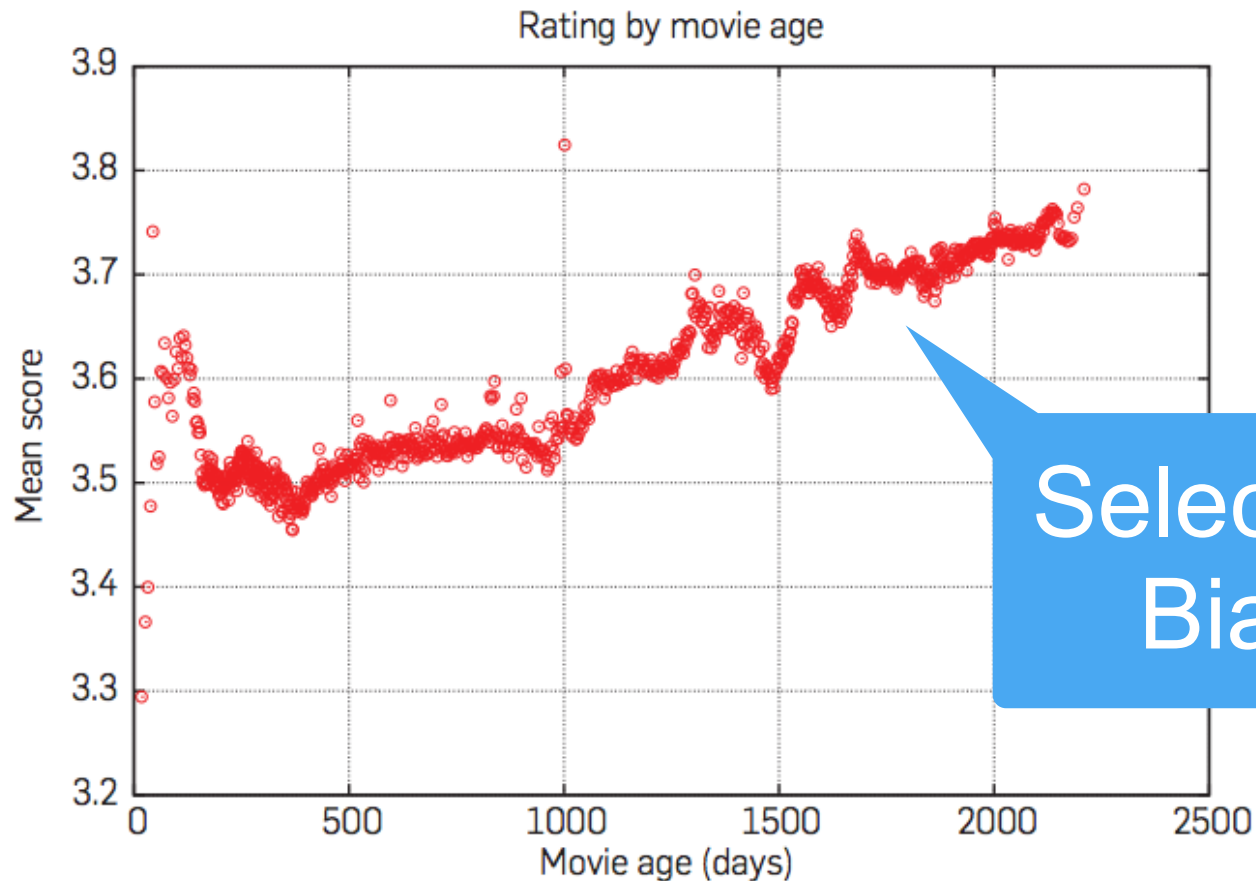
Sequence Data



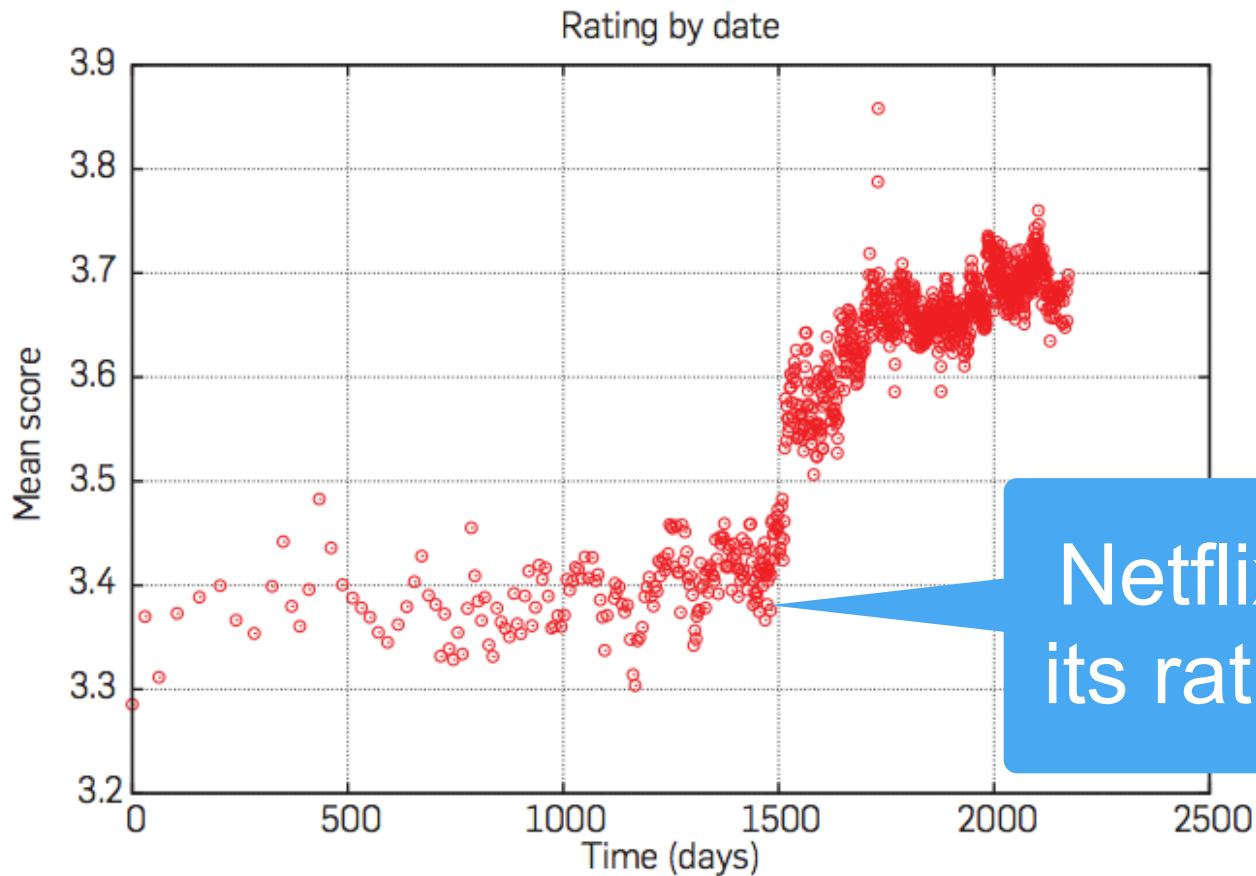
Sequential Data

- Many tasks can often be represented as varying-length sequences of fixed-length vectors. For example,
 - document: can be represented as a sequence of words
 - medical record: can often be represented as a sequence of events (encounters, medications, procedures, lab tests, diagnoses);
 - video: can be represented as a varying-length sequence of still images.
- Model outputs are sequential data: Image captioning, speech synthesis, and music generation
- Inputs are sequential data: time series prediction, video analysis, and musical information retrieval

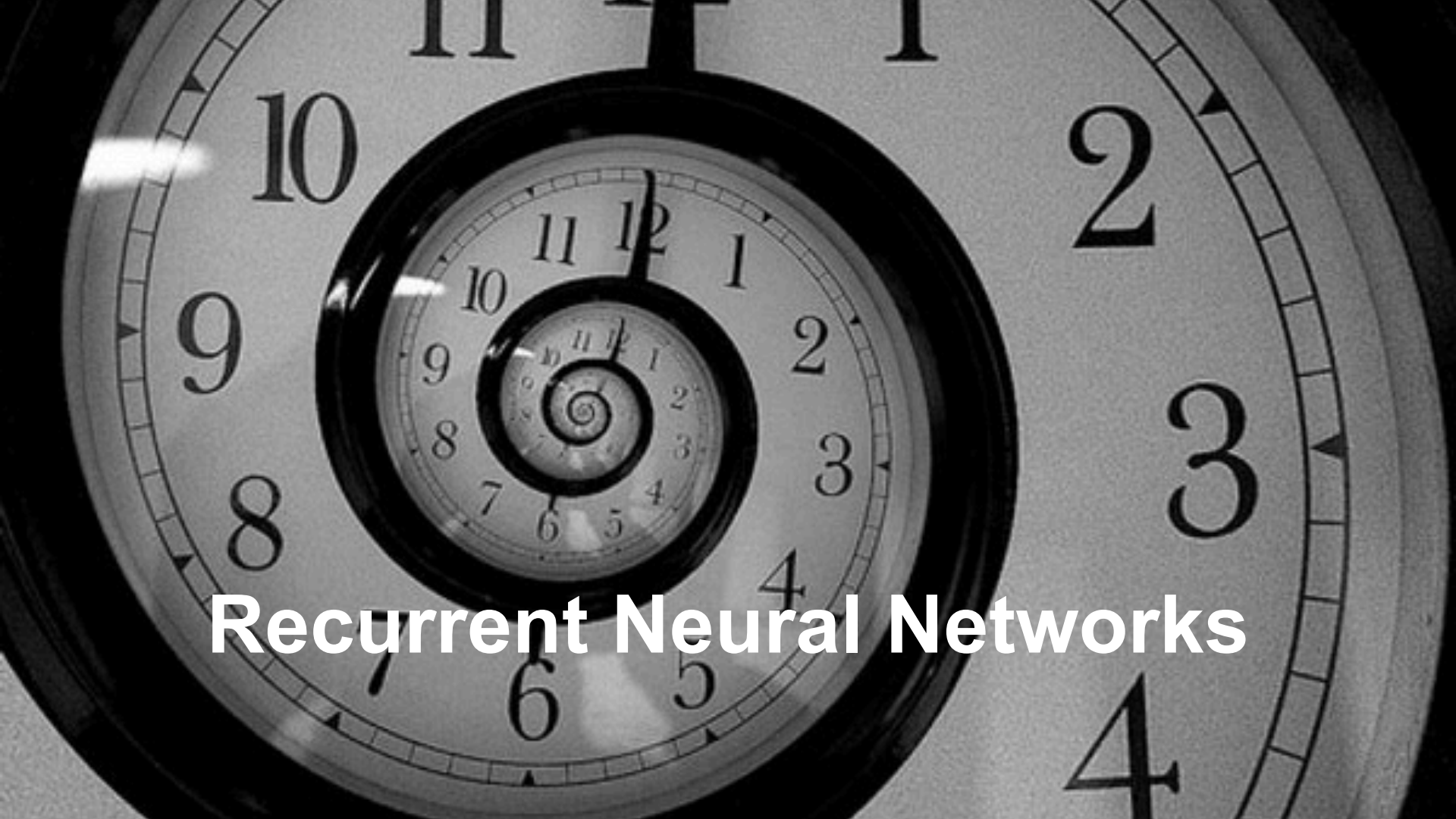
Time matters (Koren, 2009)



Time matters (Koren, 2009)



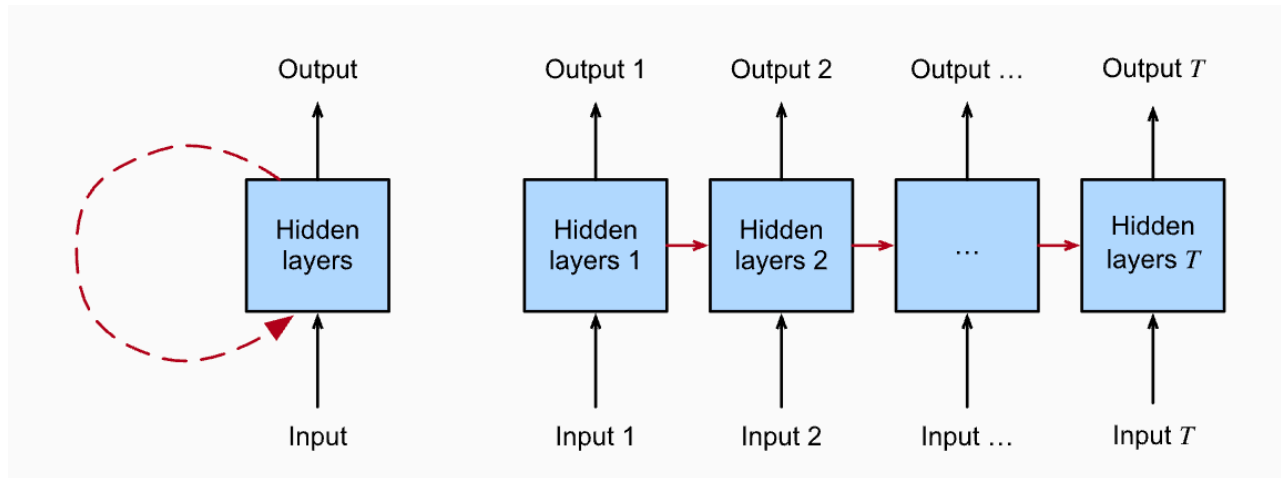
Netflix changed
its rating system

A black and white photograph of a clock face. The clock has a spiral pattern, with the numbers 1 through 12 arranged in a circular fashion. The spiral starts from the center and winds outwards, creating a sense of depth and recursion. The text "Recurrent Neural Networks" is overlaid on the lower half of the image in a white, sans-serif font.

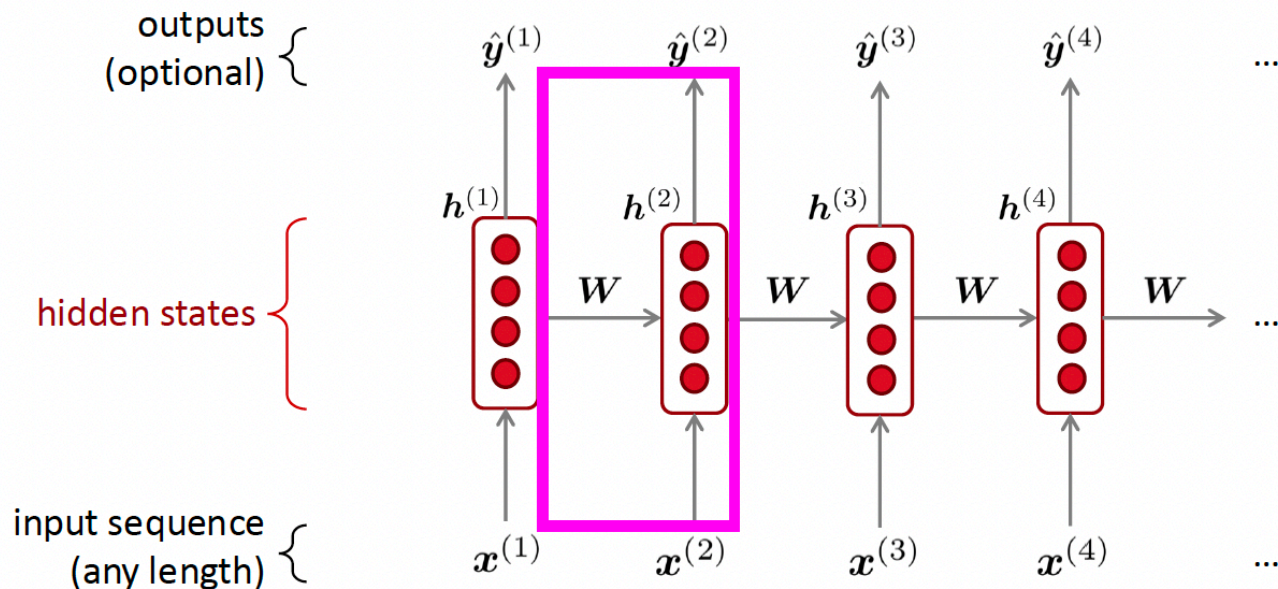
Recurrent Neural Networks

Recurrent neural networks (RNNs)

- RNNs are DL models capturing the dynamics of sequences via *recurrent* connection (cycles in the network of nodes).
- However, recurrent edges are defined in a precise way so that no ambiguity can arise.



Recurrent neural networks (RNNs)



A Simple RNN Language Model

output distribution

$$\hat{y}^{(t)} = \text{softmax}(U h^{(t)} + b_2) \in \mathbb{R}^{|V|}$$

hidden states

$$h^{(t)} = \sigma(W_h h^{(t-1)} + W_e e^{(t)} + b_1)$$

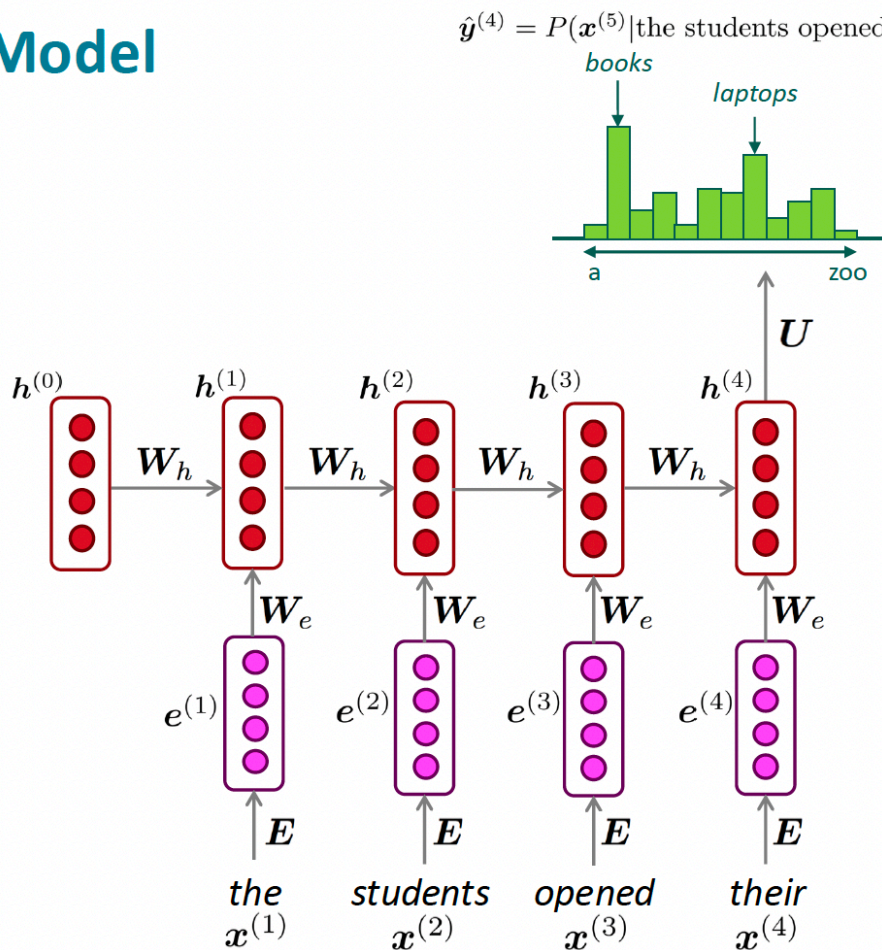
$h^{(0)}$ is the initial hidden state

word embeddings

$$e^{(t)} = E x^{(t)}$$

words / one-hot vectors

$$x^{(t)} \in \mathbb{R}^{|V|}$$



Note: this input sequence could be much longer now!

Effect of vanishing gradient on RNN-LM

- **LM task:** *When she tried to print her tickets, she found that the printer was out of toner. She went to the stationery store to buy more toner. It was very overpriced. After installing the toner into the printer, she finally printed her _____*
- *To learn from this training example, the RNN-LM needs to model the dependency between “tickets” and the target word “tickets” at the end.*
- *But if the gradient is small, the model can’t learn this dependency*
 - *So, the model is unable to predict similar long-distance dependencies at test time*

Exploding gradient

- If the gradient becomes too big, then the SGD update step becomes too big
- This can cause **bad updates**: we take too large a step and reach a weird and bad parameter configuration (with large loss)
 - You think you've found a hill to climb, but suddenly you're in Iowa
- In the worst case, this will result in **Inf** or **NaN** in your network (then you have to restart training from an earlier checkpoint)

How to fix the vanishing gradient problem?

- The main problem is that it's too difficult for the RNN to learn to preserve information over many time steps.
- In some RNNs, the hidden state is constantly being rewritten.
- First: How about an RNN with separate memory which is added to?
 - Long Short-Term Memory (LSTM) [[link](#)]
- And then: Creating more direct and linear pass-through connections in model
 - Attention, residual connections, etc.

Gradient clipping: solution for exploding gradient

- **Gradient clipping**: if the norm of the gradient is greater than some threshold, scale it down before applying SGD update

Algorithm 1 Pseudo-code for norm clipping

```
 $\hat{\mathbf{g}} \leftarrow \frac{\partial \mathcal{E}}{\partial \theta}$   
if  $\|\hat{\mathbf{g}}\| \geq threshold$  then  
     $\hat{\mathbf{g}} \leftarrow \frac{threshold}{\|\hat{\mathbf{g}}\|} \hat{\mathbf{g}}$   
end if
```

- **Intuition**: take a step in the same direction, but a smaller step
- In practice, **remembering to clip gradients is important**, but exploding gradients are an easy problem to solve

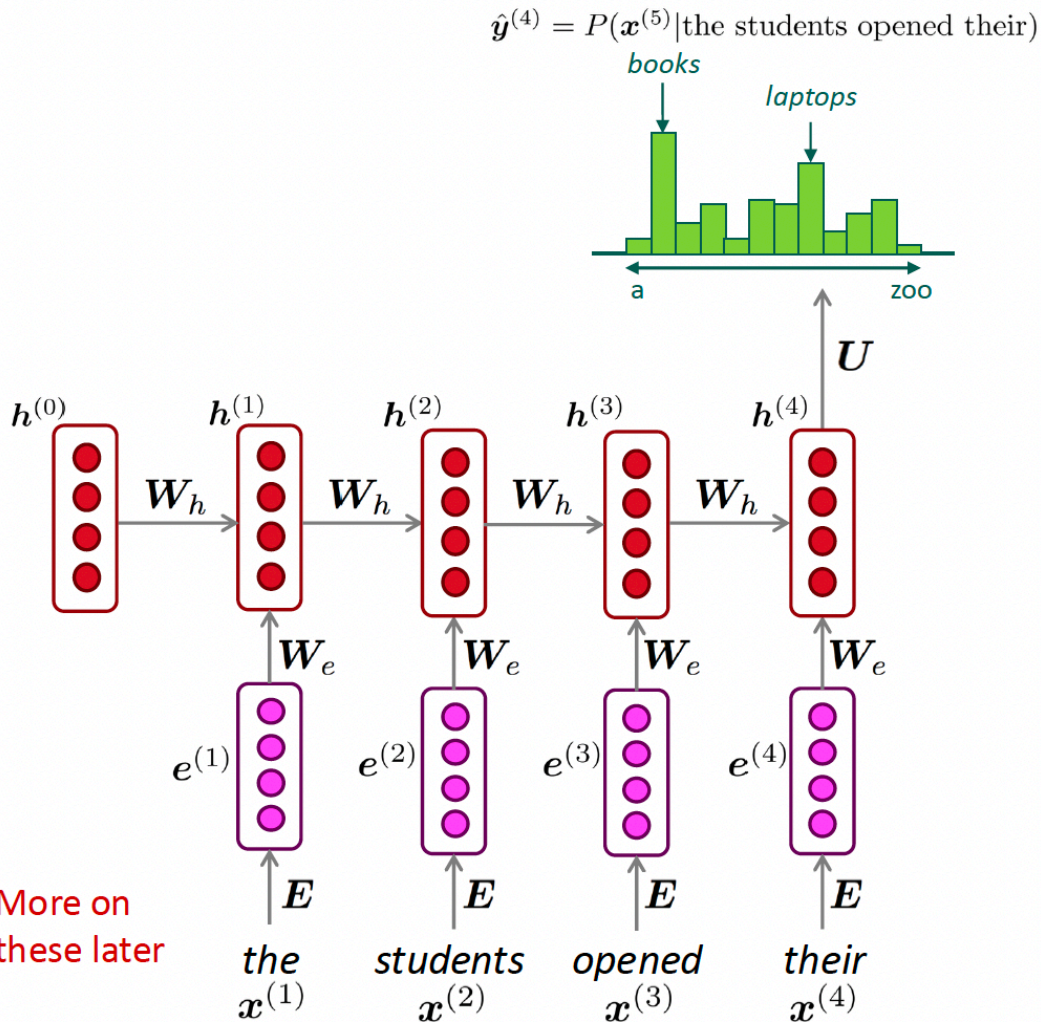
RNN Language Models

RNN Advantages:

- Can process **any length** input
- Computation for step t can (in theory) use information from **many steps back**
- **Model size doesn't increase** for longer input context
- Same weights applied on every timestep, so there is **symmetry** in how inputs are processed.

RNN Disadvantages:

- Recurrent computation is **slow**
 - In practice, difficult to access info from **many steps back**
- } More on these later



Training an RNN Language Model (Optional)

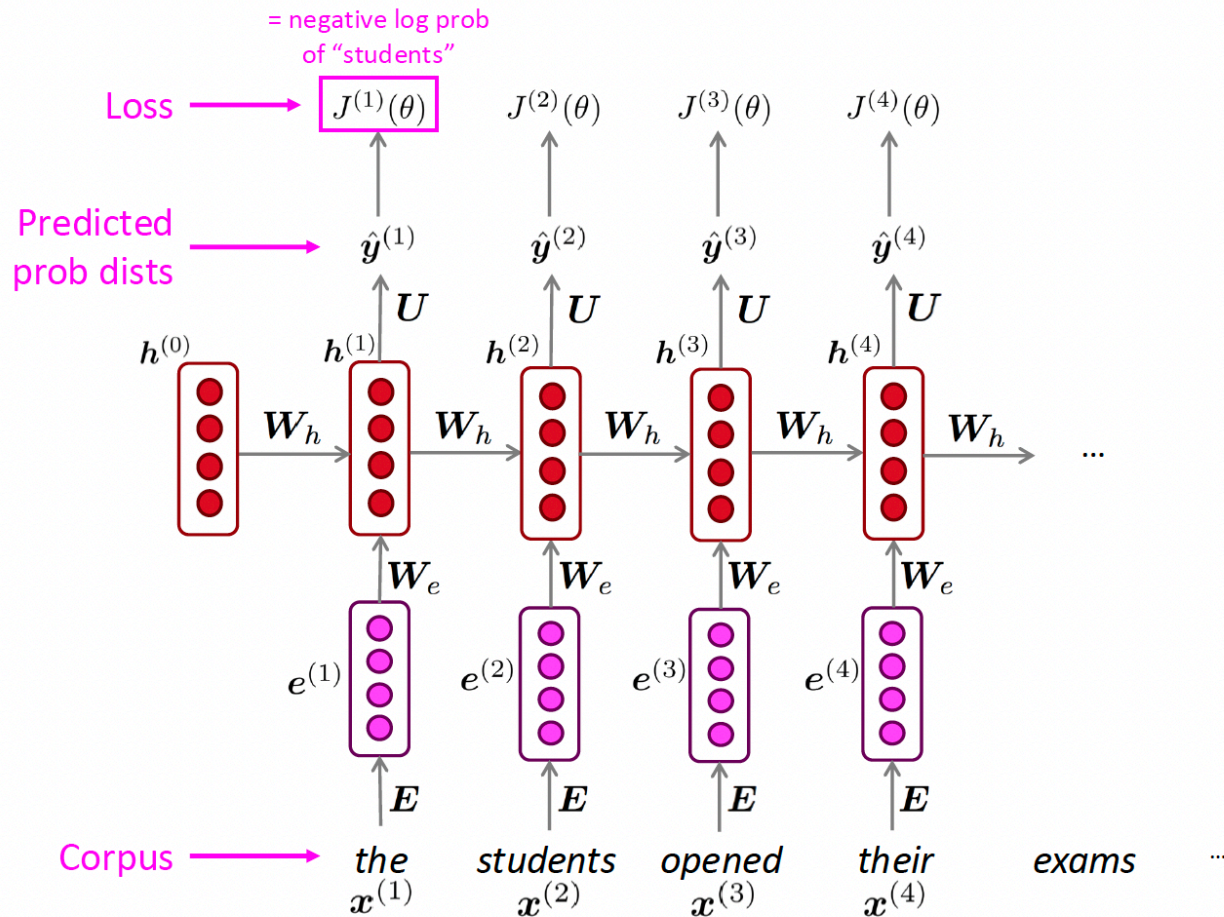
- Get a **big corpus of text** which is a sequence of words $x^{(1)}, \dots, x^{(T)}$
- Feed into RNN-LM; compute output distribution $\hat{y}^{(t)}$ **for every step t** .
 - i.e., predict probability dist of *every word*, given words so far
- **Loss function** on step t is **cross-entropy** between predicted probability distribution $\hat{y}^{(t)}$, and the true next word $y^{(t)}$ (one-hot for $x^{(t+1)}$):

$$J^{(t)}(\theta) = CE(y^{(t)}, \hat{y}^{(t)}) = - \sum_{w \in V} y_w^{(t)} \log \hat{y}_w^{(t)} = - \log \hat{y}_{x_{t+1}}^{(t)}$$

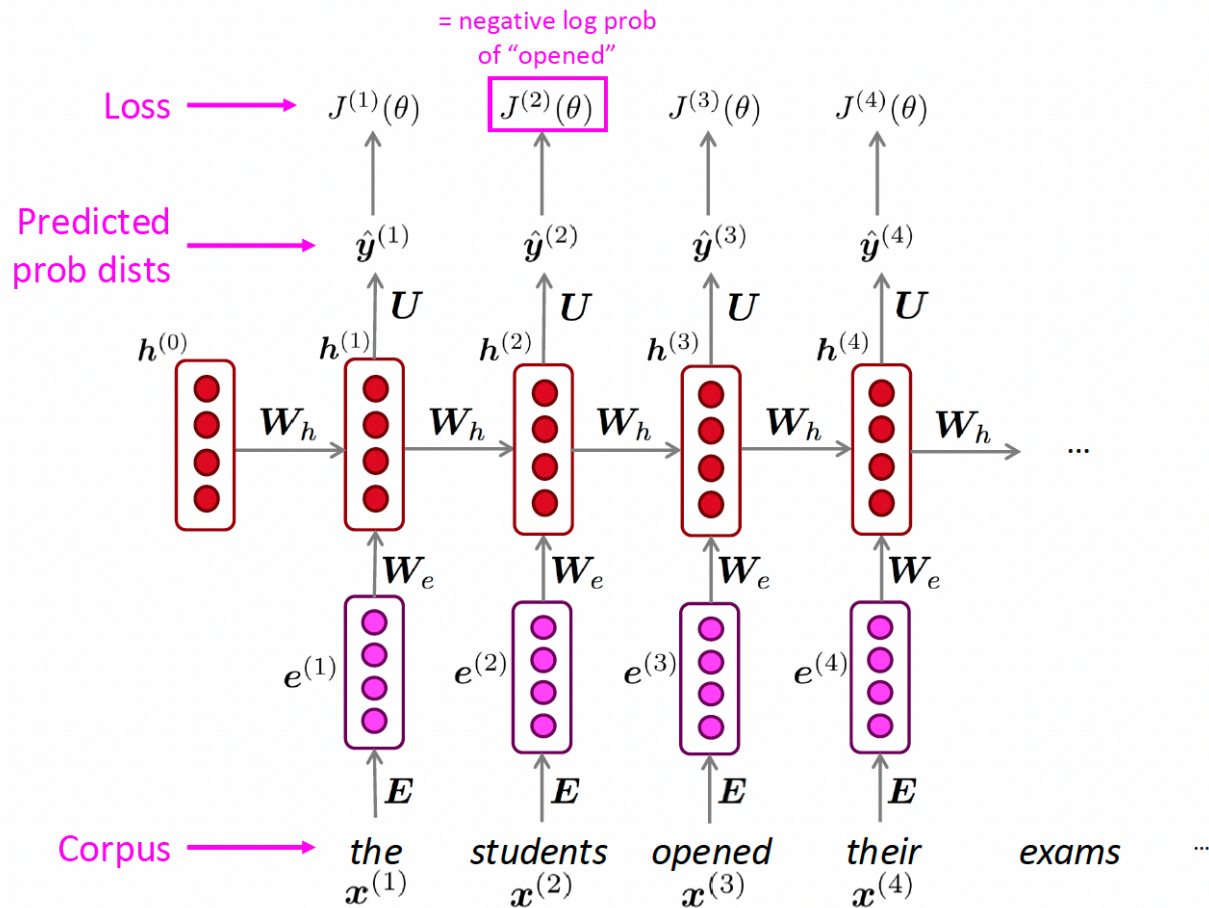
- Average this to get **overall loss** for entire training set:

$$J(\theta) = \frac{1}{T} \sum_{t=1}^T J^{(t)}(\theta) = \frac{1}{T} \sum_{t=1}^T - \log \hat{y}_{x_{t+1}}^{(t)}$$

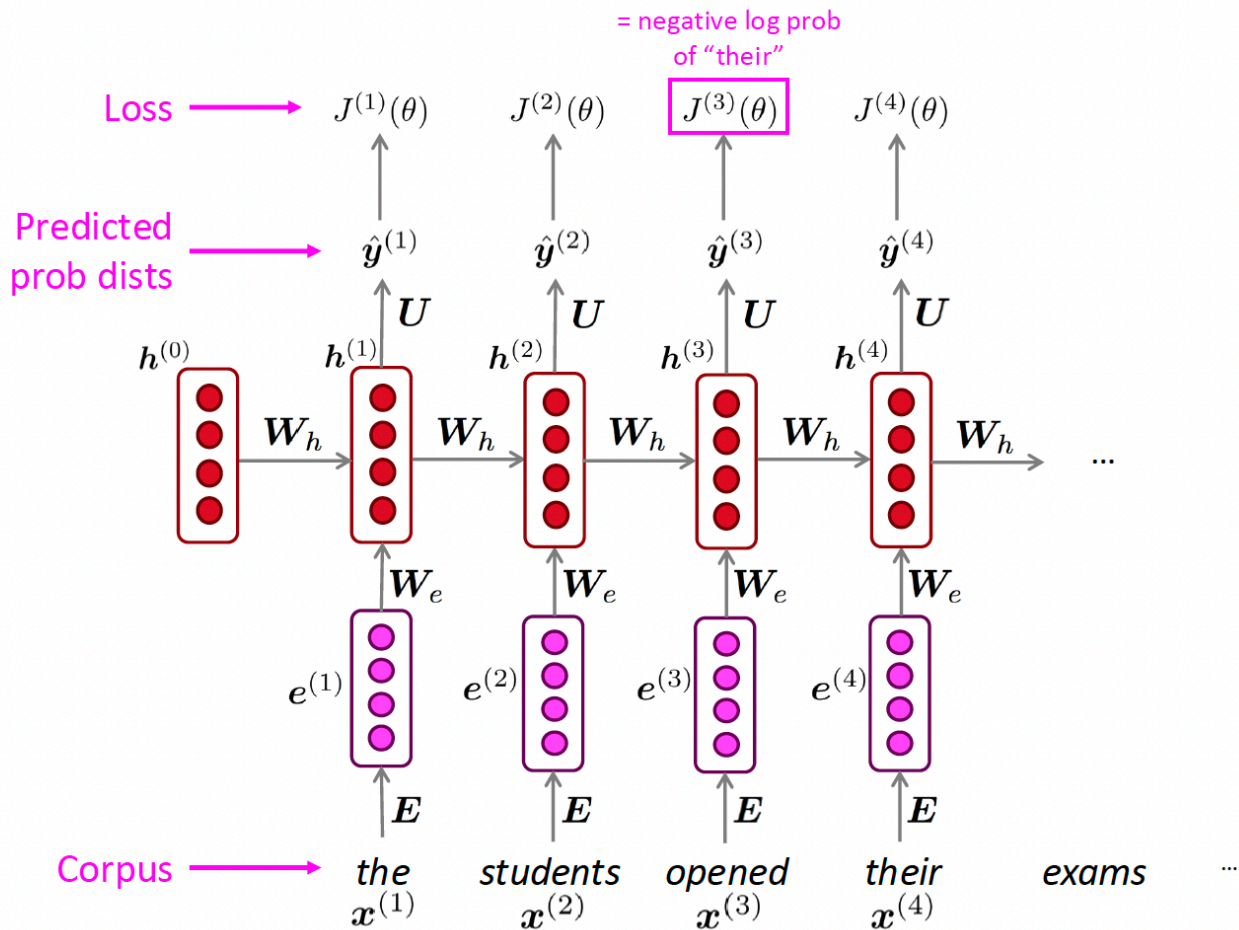
Training an RNN Language Model



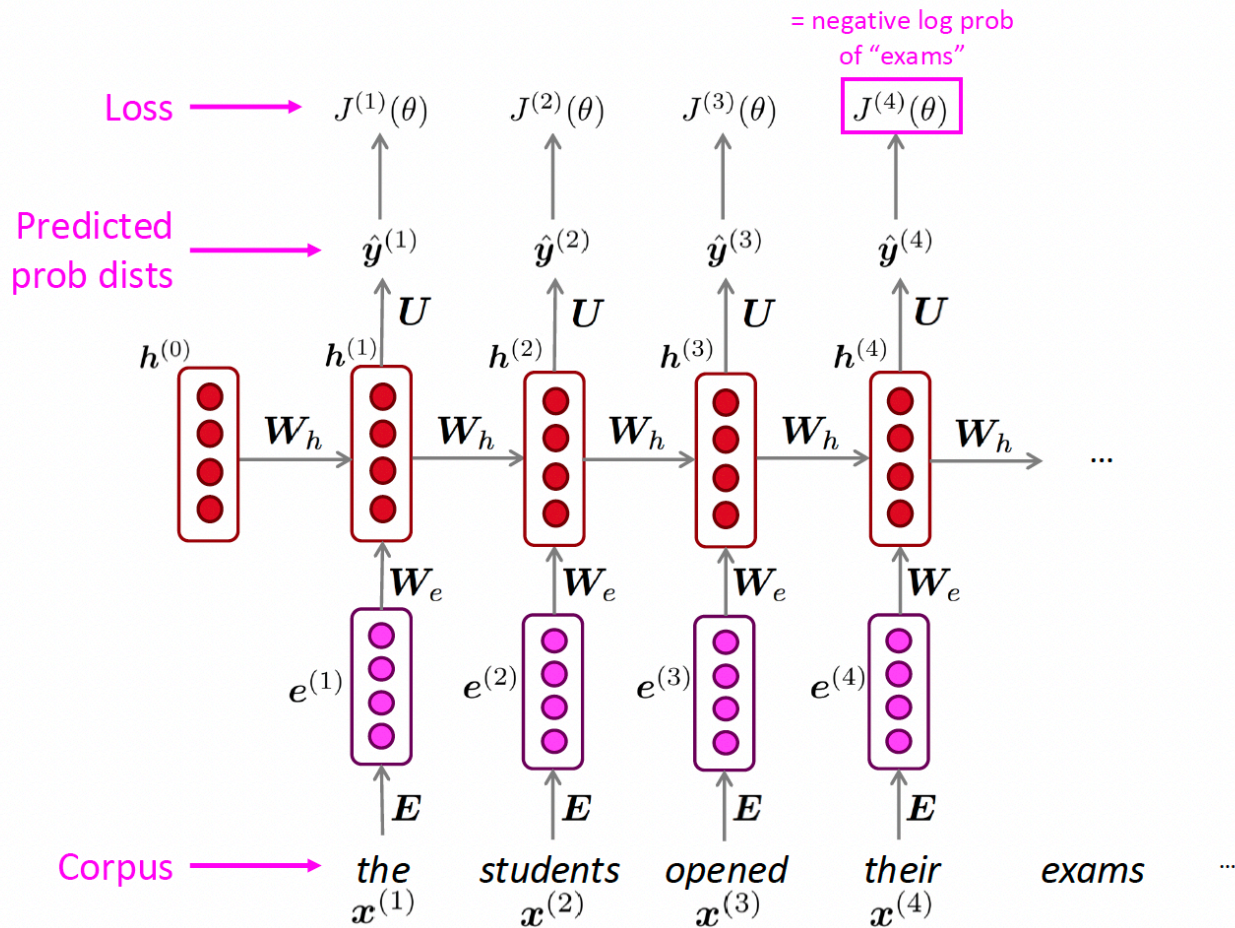
Training an RNN Language Model



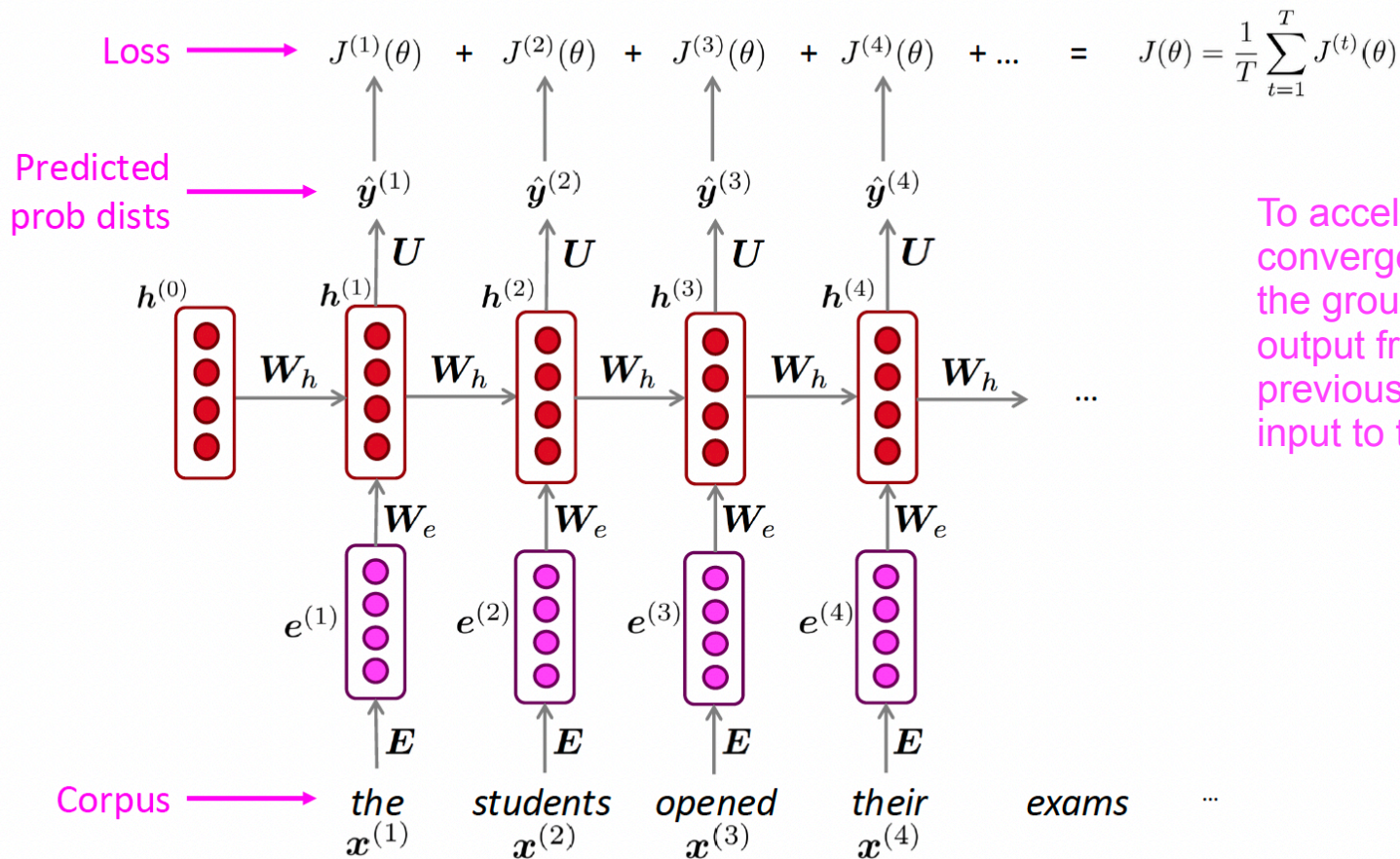
Training an RNN Language Model



Training an RNN Language Model



Training an RNN Language Model

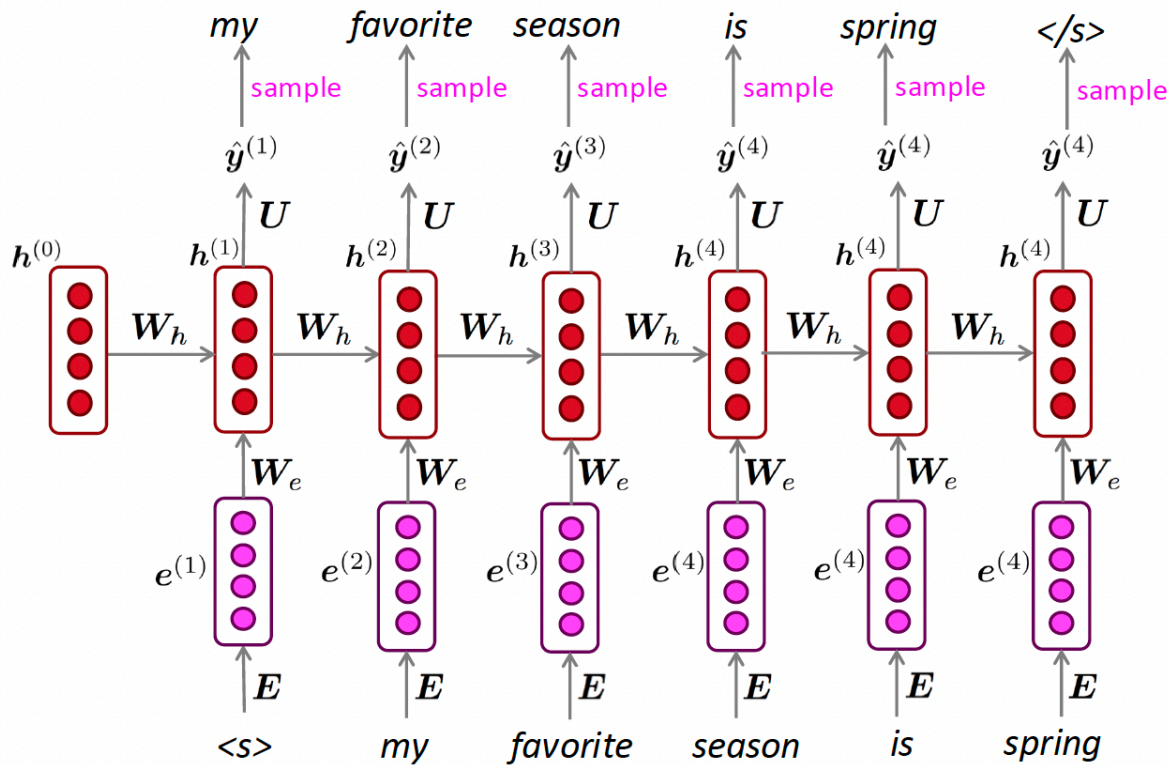


“Teacher forcing”

To accelerate convergence, feed the ground truth output from the previous time step as input to the next.

Generating with an RNN Language Model (“Generating roll outs”)

You can use a RNN Language Model to **generate text** by **repeated sampling**.
Sampled output becomes next step's input.



Generating text with an RNN Language Model

Let's have some fun!

- You can train an RNN-LM on any kind of text, then generate text in that style.
- RNN-LM trained on *Harry Potter*:



“Sorry,” Harry shouted, panicking—“I’ll leave those brooms in London, are they?”

“No idea,” said Nearly Headless Nick, casting low close by Cedric, carrying the last bit of treacle Charms, from Harry’s shoulder, and to answer him the common room perched upon it, four arms held a shining knob from when the spider hadn’t felt it seemed. He reached the teams too.

Source: <https://medium.com/deep-writing/harry-potter-written-by-artificial-intelligence-8a9431803da6>

Generating text with an RNN Language Model

Let's have some fun!

- You can train an RNN-LM on any kind of text, then generate text in that style.
- RNN-LM trained on **recipes**:

Title: CHOCOLATE RANCH BARBECUE
Categories: Game, Casseroles, Cookies, Cookies
Yield: 6 Servings

2 tb Parmesan cheese -- chopped
1 c Coconut milk
3 Eggs, beaten

Place each pasta over layers of lumps. Shape mixture into the moderate oven and simmer until firm. Serve hot in bodied fresh, mustard, orange and cheese.

Combine the cheese and salt together the dough in a large skillet; add the ingredients and stir in the chocolate and pepper.



Source: <https://gist.github.com/nylki/1efbaa36635956d35bcc>

Case Study: A Fixed-window model of RNN

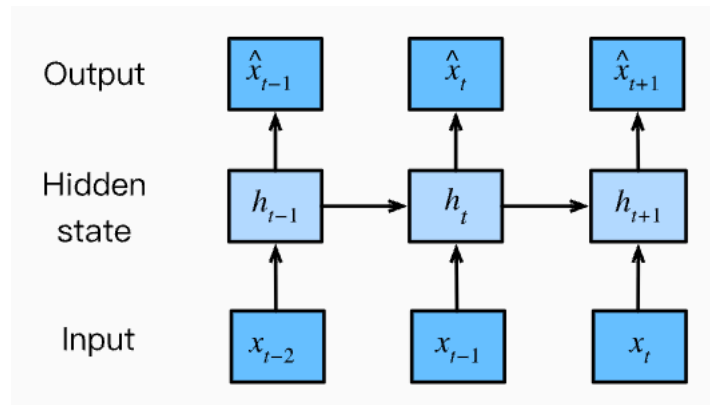


Historic Data: FTSE 100 index from 2000 to 2026



Prediction of Future Prices

- Make short-term trades, e.g., to predict the price in 2027.
- The trade wants to know the probability distribution: $P(x_{2027} | x_{2026}, x_{2025}, \dots, x_{2000})$
- To develop a model that
 - maintains a summary h_t of the past observations and
 - at the same time update h_t in addition to the prediction $\hat{x}_t$:



Construct data from training data

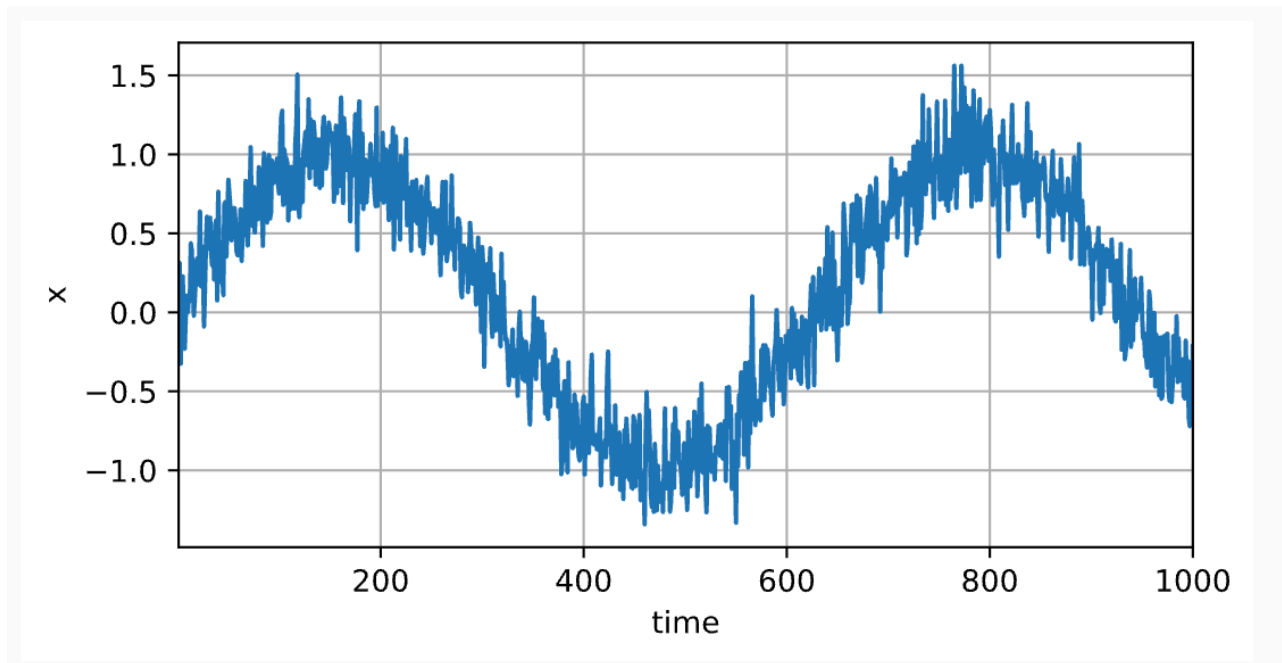
- Typically, create examples by sampling windows randomly.

```
%matplotlib inline
import torch
from torch import nn
from d2l import torch as d2l
```

```
class Data(d2l.DataModule):
    def __init__(self, batch_size=16, T=1000, num_train=600, tau=4):
        self.save_hyperparameters()
        self.time = torch.arange(1, T + 1, dtype=torch.float32)
        self.x = torch.sin(0.01 * self.time) + torch.randn(T) * 0.2
```

```
data = Data()
d2l.plot(data.time, data.x, 'time', 'x', xlim=[1, 1000], figsize=(6, 3))
```

Data visualisation

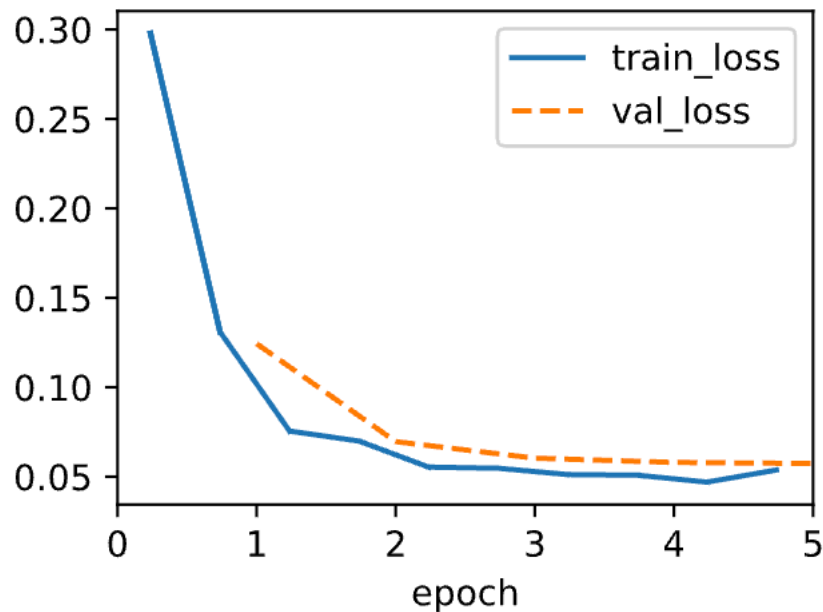


Create a data iterator

```
@d2l.add_to_class(Data)
def get_dataloader(self, train):
    features = [self.x[i : self.T-self.tau+i] for i in range(self.tau)]
    self.features = torch.stack(features, 1)
    self.labels = self.x[self.tau:].reshape((-1, 1))
    i = slice(0, self.num_train) if train else slice(self.num_train, None)
    return self.get_tensorloader([self.features, self.labels], train, i)
```

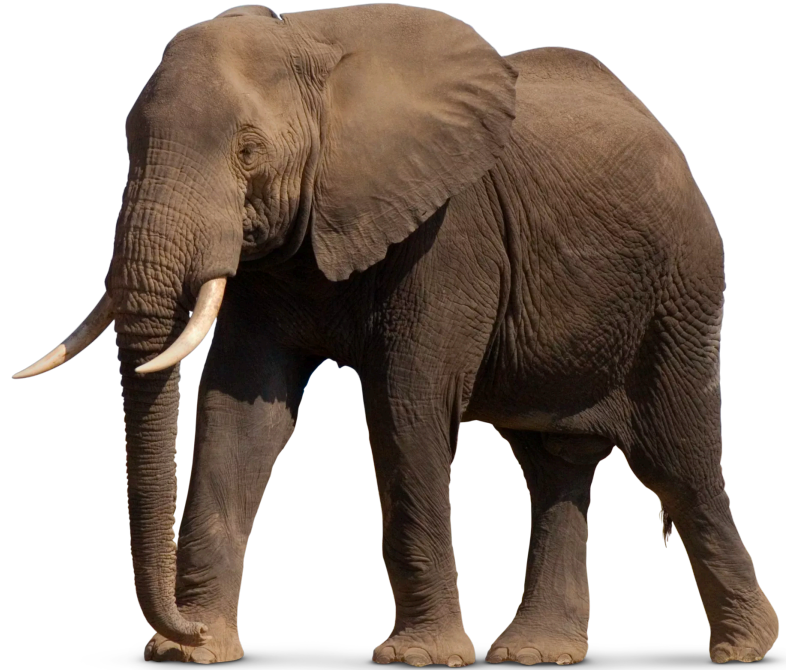
Our model will be a standard linear regression in this example:

```
model = d2l.LinearRegression(lr=0.01)
trainer = d2l.Trainer(max_epochs=5)
trainer.fit(model, data)
```

To make longer-term prediction, plug in earlier predictions as inputs.

Long Short Term Memory (LSTM)



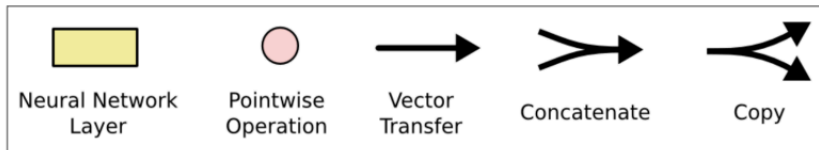
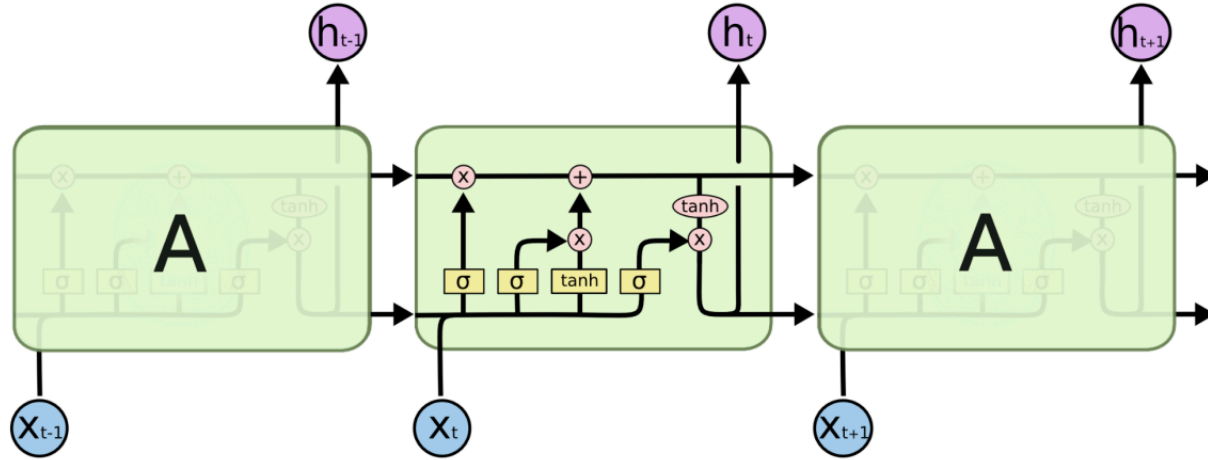
<https://colah.github.io/posts/2015-08-Understanding-LSTMs/>

Long Short-Term Memory RNNs (LSTMs)

- A type of RNN proposed by Hochreiter and Schmidhuber in 1997 as a solution to the vanishing gradients problem.
 - Everyone cites that paper but really a crucial part of the modern LSTM is from Gers et al. (2000) 
- On step t , there is a **hidden state** $h^{(t)}$ and a **cell state** $c^{(t)}$
 - Both are vectors length n
 - The cell stores **long-term information**
 - The LSTM can **read**, **erase**, and **write** information from the cell
 - The cell becomes conceptually rather like RAM in a computer
- The selection of which information is erased/written/read is controlled by three corresponding **gates**
 - The gates are also vectors length n
 - On each timestep, each element of the gates can be **open** (1), **closed** (0), or somewhere in-between
 - The gates are **dynamic**: their value is computed based on the current context

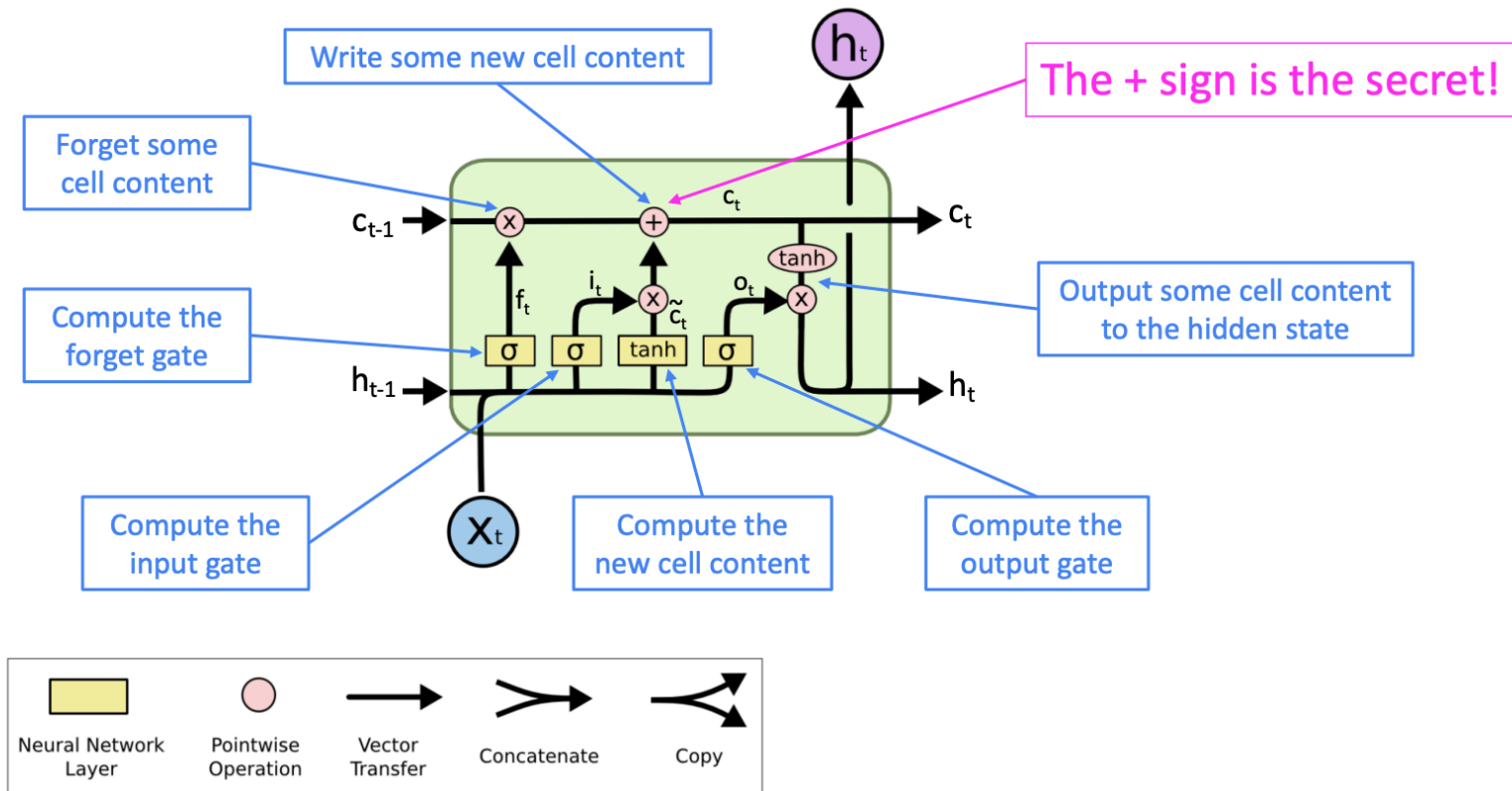
Long Short-Term Memory (LSTM)

You can think of the LSTM equations visually like this:



Long Short-Term Memory (LSTM)

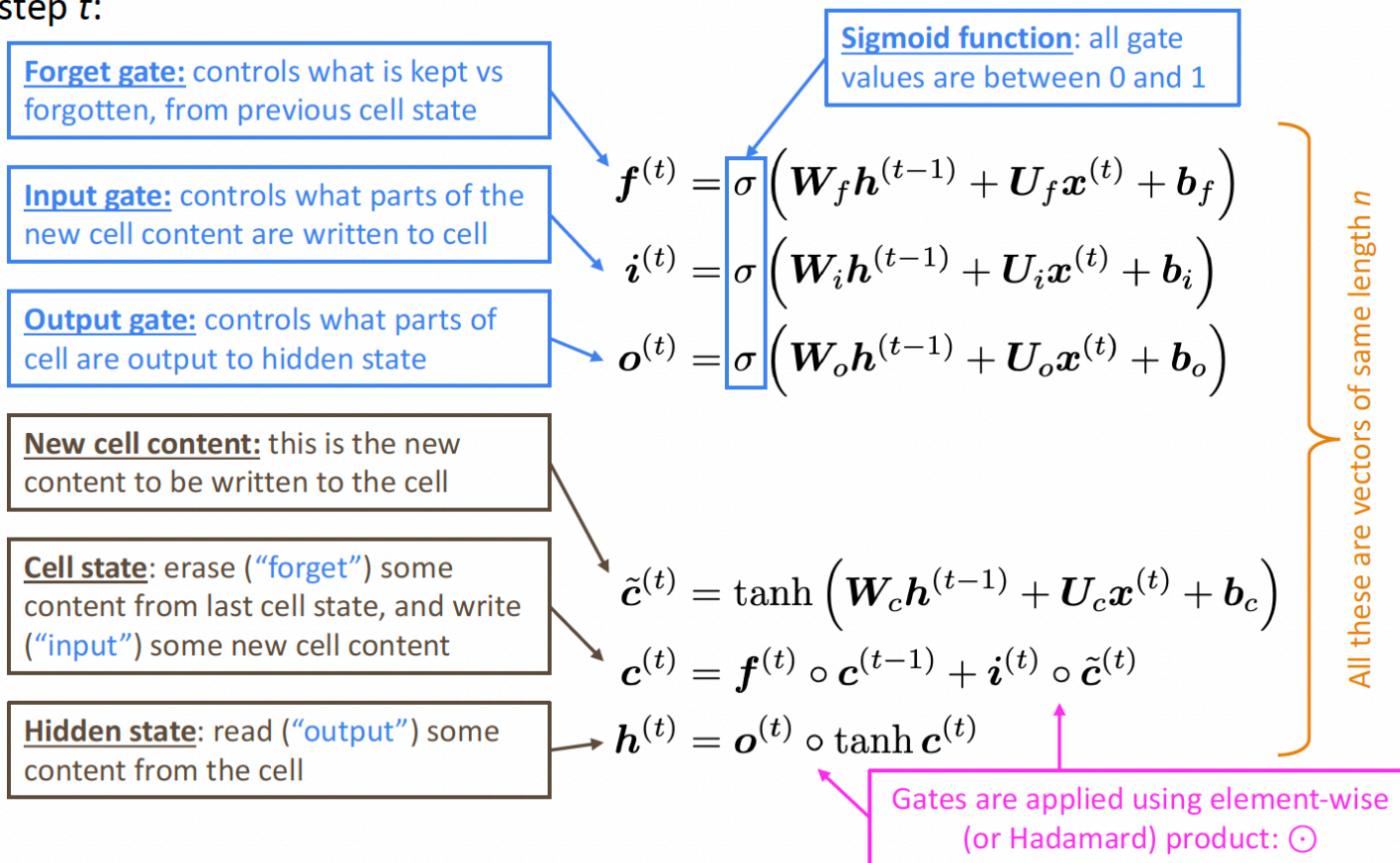
You can think of the LSTM equations visually like this:



Long Short-Term Memory (LSTM)

(Optional)

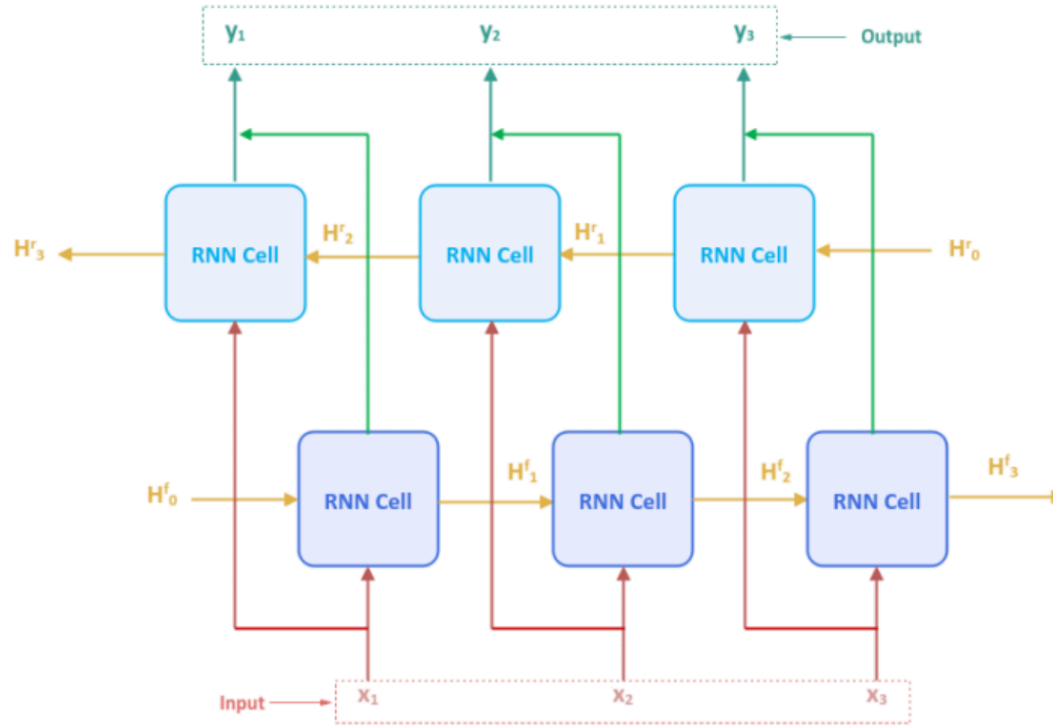
We have a sequence of inputs $x^{(t)}$, and we will compute a sequence of hidden states $h^{(t)}$ and cell states $c^{(t)}$. On timestep t :



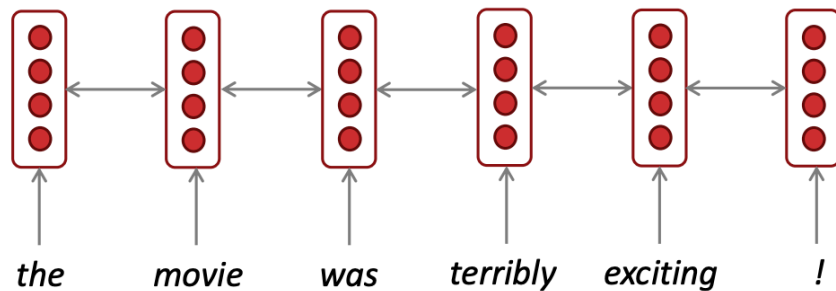
LSTMs: real-world success

- In 2013–2015, LSTMs started achieving state-of-the-art results
 - Successful tasks include handwriting recognition, speech recognition, machine translation, parsing, and image captioning, as well as language models
 - LSTMs became the dominant approach for most NLP tasks
- Now, other approaches (e.g., Transformers) have become dominant for many tasks

Bidirectional Recurrent Neural Networks (BRNN)



Bidirectional RNNs: simplified diagram



The two-way arrows indicate bidirectionality and the depicted hidden states are assumed to be the concatenated forwards+backwards states

Bidirectional RNNs

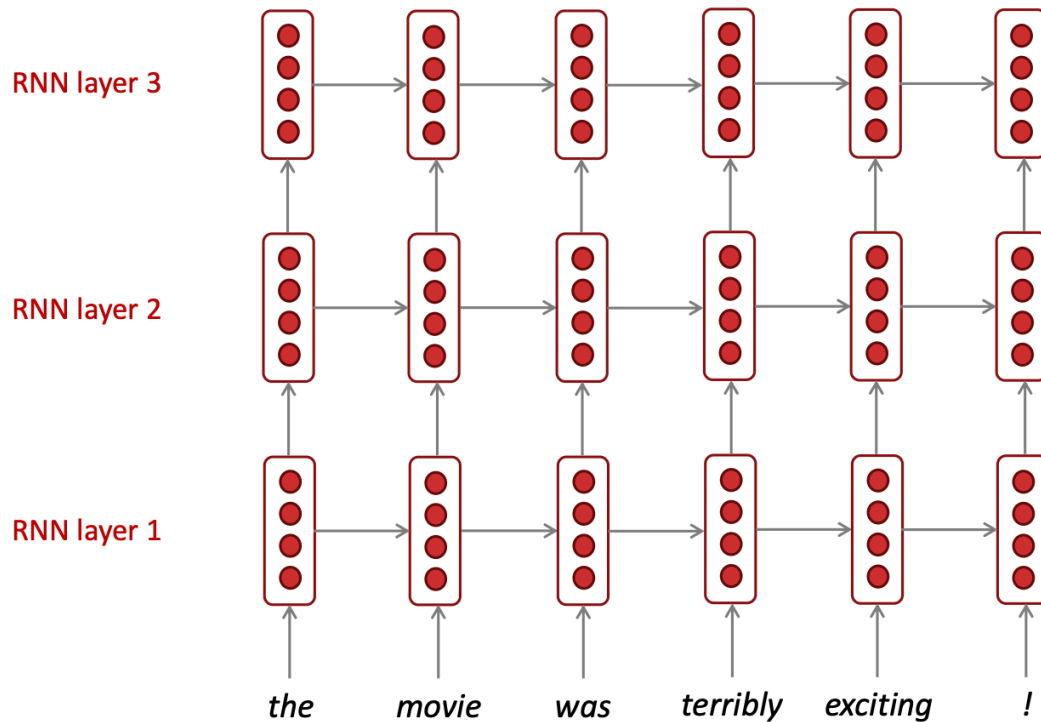
- Note: bidirectional RNNs are only applicable if you have access to the **entire input sequence**
 - They are **not** applicable to Language Modeling, because in LM you *only* have left context available.
- If you do have entire input sequence (e.g., any kind of encoding), **bidirectionality is powerful** (you should use it by default).
- For example, **BERT (Bidirectional Encoder Representations from Transformers)** is a powerful pretrained contextual representation system **built on bidirectionality**.
 - You will learn more about **transformers** include BERT in a couple of weeks!

Multi-layer RNNs

- RNNs are already “deep” on one dimension (they unroll over many timesteps)
- We can also make them “deep” in another dimension by **applying multiple RNNs** – this is a multi-layer RNN.
- This allows the network to compute **more complex representations**
 - The **lower RNNs** should **compute lower-level features** and the **higher RNNs** should compute **higher-level features**.
- Multi-layer RNNs are also called ***stacked RNNs***.

Multi-layer RNNs

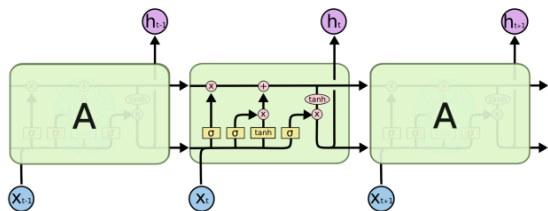
The hidden states from RNN layer i are the inputs to RNN layer $i+1$



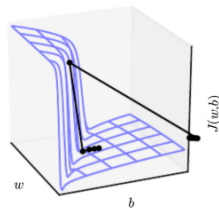
Multi-layer RNNs in practice

- High-performing RNNs are often multi-layer (but aren't as deep as convolutional or feed-forward networks)
- For example: In a 2017 paper, Britz et al find that for Neural Machine Translation, 2 to 4 layers is best for the encoder RNN, and 4 layers is best for the decoder RNN
 - Usually, skip-connections/dense-connections are needed to train deeper RNNs (e.g., 8 layers)
- Transformer-based networks (e.g., BERT) are usually deeper, like 12 or 24 layers.
 - You will learn about Transformers later; they have a lot of skipping-like connections

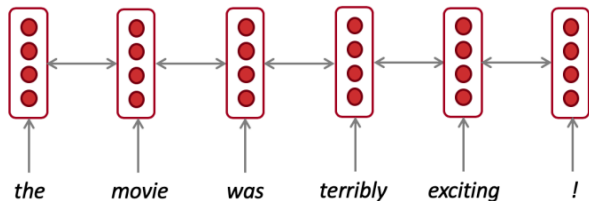
Summary



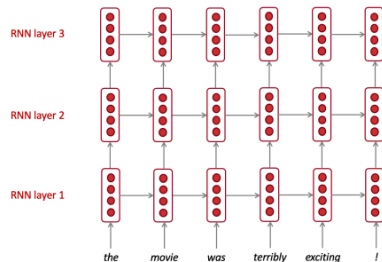
1. LSTMs are powerful



2. Clip your gradients



3. Use bidirectionality when possible



4. Multi-layer RNNs are more powerful, but you might need skip connections if it's deep